ML-10 Object Detection II

การตรวจจับแมวและสุนัข (Cat & Dog Detection) ด้วย YOLOv8

จักรพัทศ์ ยอดระยับ · วิศวกรรมคอมพิวเตอร์ มหาวิทยาลัยเทคโนโลยีราชมงคลธัญบุรี · ตุลาคม 2569

0.930	0.781	99.0%	~47 ms
mAP@0.5 (test)	mAP@0.5:0.95	ความแม่นยำระดับภาพ	ต่อภาพ บน CPU

1. วัตถุประสงค์

ฝึกโมเดล Object Detection แบบ **one-stage** (YOLOv8) ให้ระบุทั้ง**ตำแหน่ง** (bounding box) และ**ชนิด** (cat/dog) ของสัตว์ในภาพ แล้วประเมินผลด้วย Precision, Recall และ mAP พร้อมทดสอบการใช้งานกับรูปภาพและแบบ real-time ต่อยอดจาก ML-09 ที่ใช้ two-stage detector (R-CNN, Fast R-CNN)

2. ทฤษฎีโดยสังเขป

Two-stage (R-CNN, Fast/Faster R-CNN) สร้าง region proposal ก่อนแล้วจึงจำแนกแต่ละ region ส่วน **One-stage** (YOLO, SSD) แบ่งภาพเป็นกริดและทำนาย class + box พร้อมกันใน forward pass เดียว จึงเร็วกว่ามาก

YOLOv8 ประกอบด้วย **Backbone** (CSPDarknet + C2f) สกัด feature, **Neck** (PAN-FPN) รวม feature หลายสเกล และ **Head** แบบ anchor-free ที่แยก branch สำหรับ classification และ box regression. Loss = box loss (CIoU) + classification loss (BCE) + DFL (Distribution Focal Loss)

IoU = พื้นที่ซ้อนทับ / พื้นที่รวมของกล่องทำนายกับกล่องจริง; **mAP@0.5** คือค่าเฉลี่ย Average Precision ที่ IoU ≥ 0.5 และ **mAP@0.5:0.95** คือค่าเฉลี่ยที่ IoU 0.50–0.95 (ขั้นละ 0.05) ซึ่งเข้มงวดเรื่องความแม่นของกรอบมากกว่า

3. ชุดข้อมูล

Split	cat	dog	รวม	ที่มา
train	420	420	840	Data/training (85%)
val	74	74	148	Data/training (15%)
test	100	100	200	Data/testing

ตัดภาพซ้ำออก 12 ภาพ (ไฟล์ `(1).jpg` ที่ MD5 ตรงกับต้นฉบับ) และแบ่งแบบ stratified (seed 42)

การสร้าง label (auto-labeling)

ชุดข้อมูลมีเพียงชื่อ class ในชื่อไฟล์ ไม่มี bounding box จึงใช้ YOLO11-L (pre-trained COCO) เป็น "teacher" หากรอบสัตว์ที่ conf ≥ 0.25 โดยกำหนด class ตามชื่อไฟล์เสมอ หากหาไม่พบใช้กรอบเต็มภาพ ได้ทั้งหมด 1,365 กล่อง

label_source	จำนวน	ความหมาย
auto_yolo	1,333	teacher หาเจอ และ class ตรงกับชื่อไฟล์
auto_yolo_classfix	15	teacher หาเจอแต่ทาย class ผิด → แก้ตามชื่อไฟล์
full_image_fallback	17	teacher หาไม่เจอ → ใช้กรอบเต็มภาพ



รูปที่ 1 ตัวอย่าง ground-truth bounding box ในชุด train

> ข้อจำกัด: label มาจากโมเดล ไม่ได้วาดด้วยมือ ค่า mAP จึงสะท้อนความสอดคล้องกับ teacher model ซึ่งอาจคลาดเคลื่อนจากกรอบจริงเล็กน้อย

4. วิธีการทดลอง

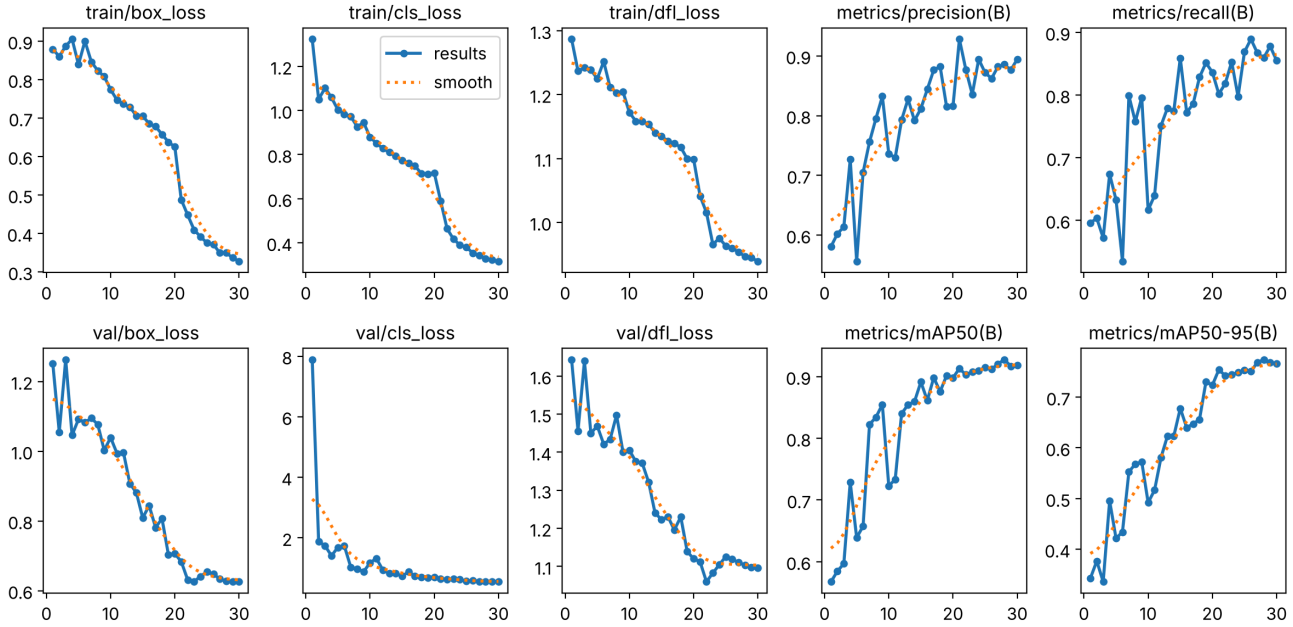
Hyperparameter	ค่า
Model	YOLOv8n (3.0 M parameters, 8.6 GFLOPs) pre-trained COCO
Image size / Batch / Epochs	416 × 416 / 16 / 30 (early-stopping patience 10)
Optimizer	AdamW, lr0 = 0.001, cosine learning-rate schedule
Augmentation	Mosaic, HSV, horizontal flip, scale/translate (ปิด Mosaic 10 epoch สุดท้าย)
Hardware	CPU 2 cores (~170 วินาที/epoch, รวม ~85 นาที)

ลำดับไฟล์: `prepare_dataset.py` → `train.py` → `evaluate.py` → `image_test.py` / `realtime_test.py`

5. ผลการทดลอง

5.1 การเทรน (validation set)

Epoch	box loss	cls loss	Precision	Recall	mAP@0.5	mAP@0.5:0.95
1	0.879	1.324	0.581	0.596	0.568	0.343
5	0.839	1.003	0.556	0.633	0.639	0.422
10	0.774	0.880	0.736	0.616	0.723	0.492
15	0.705	0.774	0.812	0.859	0.891	0.678
20	0.625	0.718	0.816	0.836	0.898	0.724
25	0.375	0.380	0.872	0.869	0.915	0.753
28 (best)	0.350	0.328	0.886	0.860	**0.928**	**0.774**
30	0.327	0.316	0.894	0.855	0.919	0.766



รูปที่ 2 loss และ metrics ในแต่ละ epoch (train loss ลดลงชัดหลัง epoch 20 เพราะปิด Mosaic augmentation)

5.2 Test set

Class	Precision	Recall	mAP@0.5	mAP@0.5:0.95
all	0.936	0.884	**0.930**	**0.781**
cat	0.930	0.923	0.964	0.799
dog	0.943	0.845	0.895	0.762

ความเร็ว: preprocess 1.8 ms + inference 47.4 ms + postprocess 0.3 ms ต่อภาพ (CPU) ≈ 20 FPS

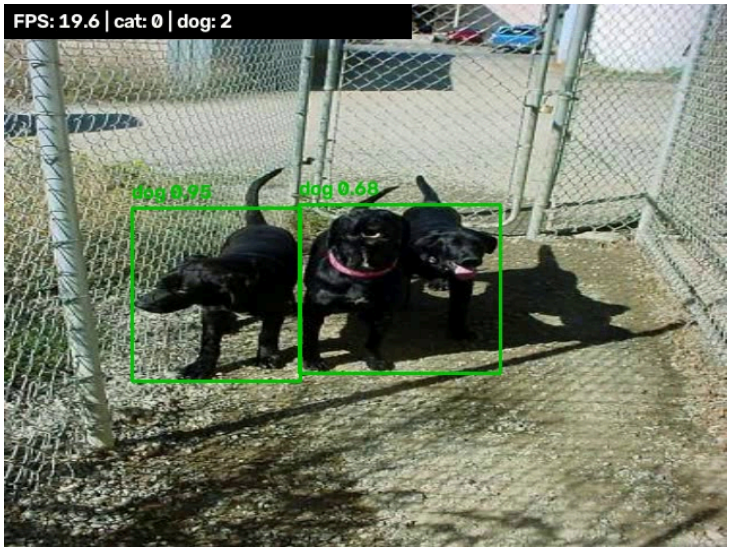
5.3 ผลระดับภาพ (conf ≥ 0.25, เลือกกล่องที่มั่นใจที่สุด)

จริง \ ทำนาย	cat	dog	ถูกต้อง
cat	100	0	100%
dog	2	98	98%

ทั้ง 200 ภาพมีการตรวจพบอย่างน้อย 1 กล่อง; จาก 239 กล่องที่ทำนาย มี 5 กล่องที่ class ผิด (เช่น `dog.4987.jpg` ถูกทายเป็น cat ที่ conf 0.95)



รูปที่ 3 ตัวอย่างผลการตรวจจับบน test set (สีฟ้า = cat, สีเขียว = dog)



รูปที่ 4 ตัวอย่างเฟรมจาก realtime_test.py (แสดง FPS และจำนวนที่พบ)

6. วิเคราะห์และอภิปราย

- **Transfer learning ได้ผลดี:** mAP@0.5 ขึ้นจาก 0.57 เป็น 0.93 ภายใน 30 epoch ด้วยข้อมูลเพียง 840 ภาพ เพราะ backbone ที่เทรนบน COCO รู้จัก feature ของสัตว์อยู่แล้ว
- **cat ดีกว่า dog** ด้าน recall (0.92 เทียบ 0.85): สุนัขมีความหลากหลายของสายพันธุ์/ขนาด/ท่าทางสูง และภาพสุนัขมักมีหลายตัวหรือถูกตัดขอบ
- **ข้อผิดพลาดหลัก:** สุนัขขนยาวหรือหน้าแบนที่ดูคล้ายแมว และกรอบซ้อนกันเมื่อสัตว์ถูกบังบางส่วน
- **เทียบกับ ML-09:** R-CNN ต้องรัน CNN กับ ~2,000 proposals ต่อภาพ ส่วน YOLOv8n ใช้ forward pass เดียว (~47 ms บน CPU) จึงใช้งาน real-time ได้
- ค่า mAP@0.5:0.95 (0.78) ต่ำกว่า mAP@0.5 เพราะ metric นี้เข้มงวดเรื่องความแม่นของขอบกล่อง ซึ่งได้รับผลจาก label อัตโนมัติด้วย

7. สรุป

YOLOv8n ที่ fine-tune บนชุด Cat/Dog ได้ mAP@0.5 = 0.930 และ mAP@0.5:0.95 = 0.781 บน test set, จำแนกระดับภาพถูก 99% และทำงานได้ราว 20 FPS บน CPU เหมาะกับงาน real-time. แนวทางพัฒนา: ตรวจแก้ label ด้วยมือ (LabelImg/Roboflow), ใช้โมเดลใหญ่ขึ้น (YOLOv8s/m), เทรนที่ 640 px บน GPU และเพิ่มจำนวน epoch

8. เอกสารอ้างอิง

1. Redmon, J. et al. (2016). You Only Look Once: Unified, Real-Time Object Detection. CVPR.
2. Jocher, G. et al. Ultralytics YOLOv8. https://docs.ultralytics.com
3. Girshick, R. (2015). Fast R-CNN. ICCV.
4. Kaggle. Dogs vs. Cats dataset.